Figure 10-4 where the length of the path to every leaf is the same. It would be better to measure the heights directly.

Say you'd like to measure the difference in depth of the two sides of each tree. Because you want to measure it at every subtree, it makes more sense to measure from the subtree root down to its leaf nodes. That way, you can ignore where the subtree lies within the overall tree; its position with respect to the overall tree's root doesn't change the metric.

The number of nodes on the longest path from a particular node X to a leaf node is called the **height** of the subtree rooted at node X, or more simply, the height of X. Figure 10-5 shows the distinction between the level (or depth) of a node and the height of the subtree rooted at a node. In the sample tree, each subtree's height is shown in orange next to the root node of the subtree. All the leaf nodes are at height 1 regardless of their depth. The height measures the longest path below the node (to an empty child), and the level measures the distance to the tree root.

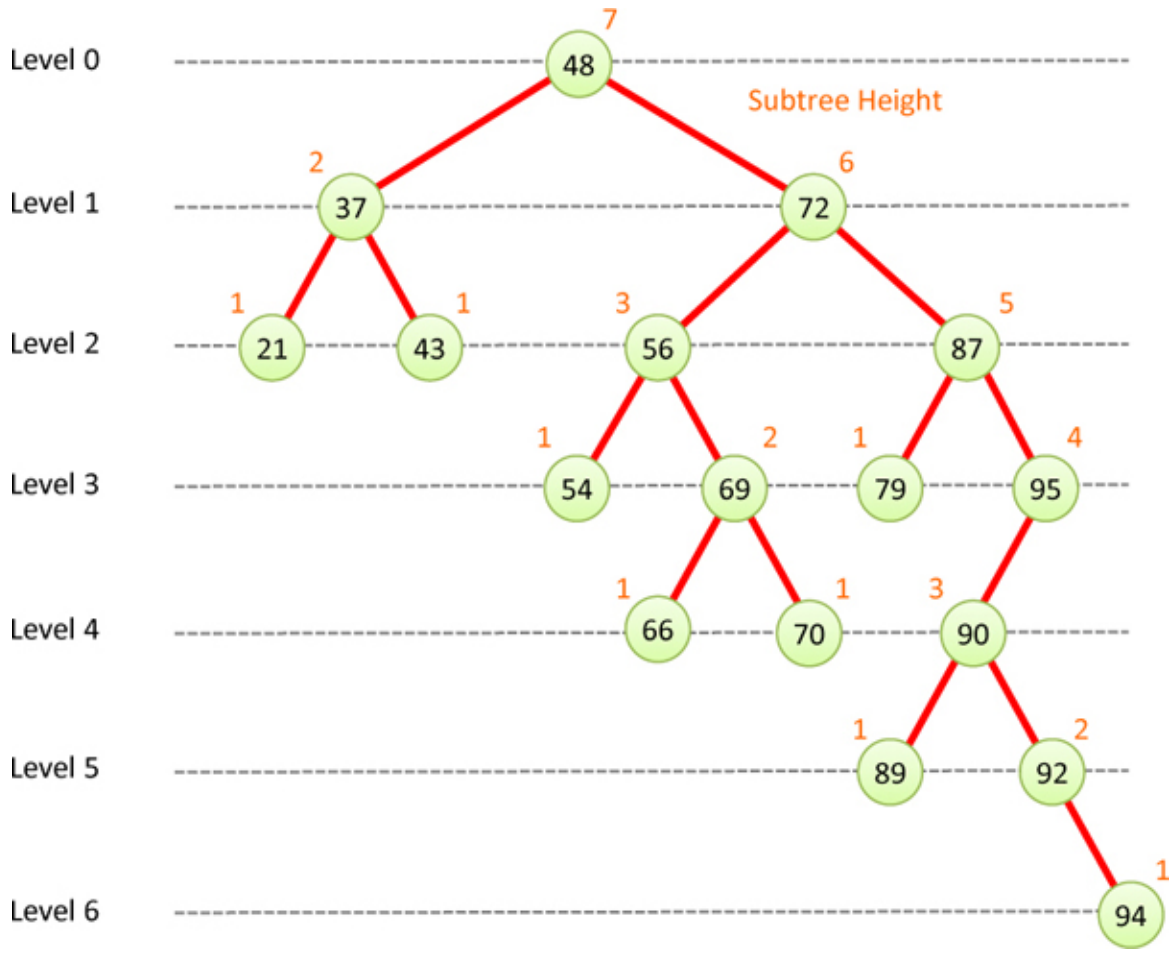

Figure 10-5 *Node levels and subtree heights*

For balance, you want to measure the difference in height between the two child branches of a node. You can subtract the right child's height from the left child's height to get the difference. When a node has only one child like nodes 17, 21, 65, and 80 do in Figure 10-4, the height of the empty child is considered zero. That way the height difference is either +1 or −1 for a node having a single leaf node as a child, like nodes 17 and 21. You want to ensure that the imbalance of those nodes is counted.

Totaling up the absolute values of the height differences gives a different overall tree metric. For the unbalanced tree in Figure 10-3, the total is 10 (three each for nodes 27 and 65 at level 1, two each for nodes 16 and 93 at level 2, and zero for all the others). The tree in Figure 10-4 would have a total of 4 (one for each of nodes 17, 21, 65, and 80). This overall metric is the same as the total of node count differences. Both find that there are 4 nodes with imbalance of one between their left and right sides.

How Much Is Unbalanced?

With the overall metrics we've described so far, a tree with a metric of zero would certainly be balanced. If you demand that every tree with a nonzero metric be considered unbalanced, then you could only have balanced trees with node counts of 1, 3, 7, 15, …, $2^N - 1$. The reason is that the only way to be perfectly balanced is to have every node link to exactly two (or no) children and completely fill all the lowest levels of the tree (the ones nearest the root).

Defining balance to be metric = 0 is not a very practical definition because you'd like the binary trees to behave something like the 2-3-4 trees, which can contain any number of nodes and still be considered balanced. Balancing binary trees is about keeping the maximum height of any node to a minimum so that the longest search path is no longer than necessary. That's exactly the case in Figure 10-4, so that tree's metric should come out balanced. Let's see how far we need to extend the definition of balanced.

Consider the smallest binary trees. An empty tree and a tree with a single node are balanced. When a tree has two nodes, one node must be at a different height than the other. That means the root node must have at least a height difference of one. So, you must allow at least one node to have height difference of +/− 1 in a balanced tree.

When you add a third node, there are five possible tree shapes, as shown in Figure 10-6. The middle one is obviously balanced. In all the others, the root node has a height difference of two, and the midlevel node has a height difference of one.

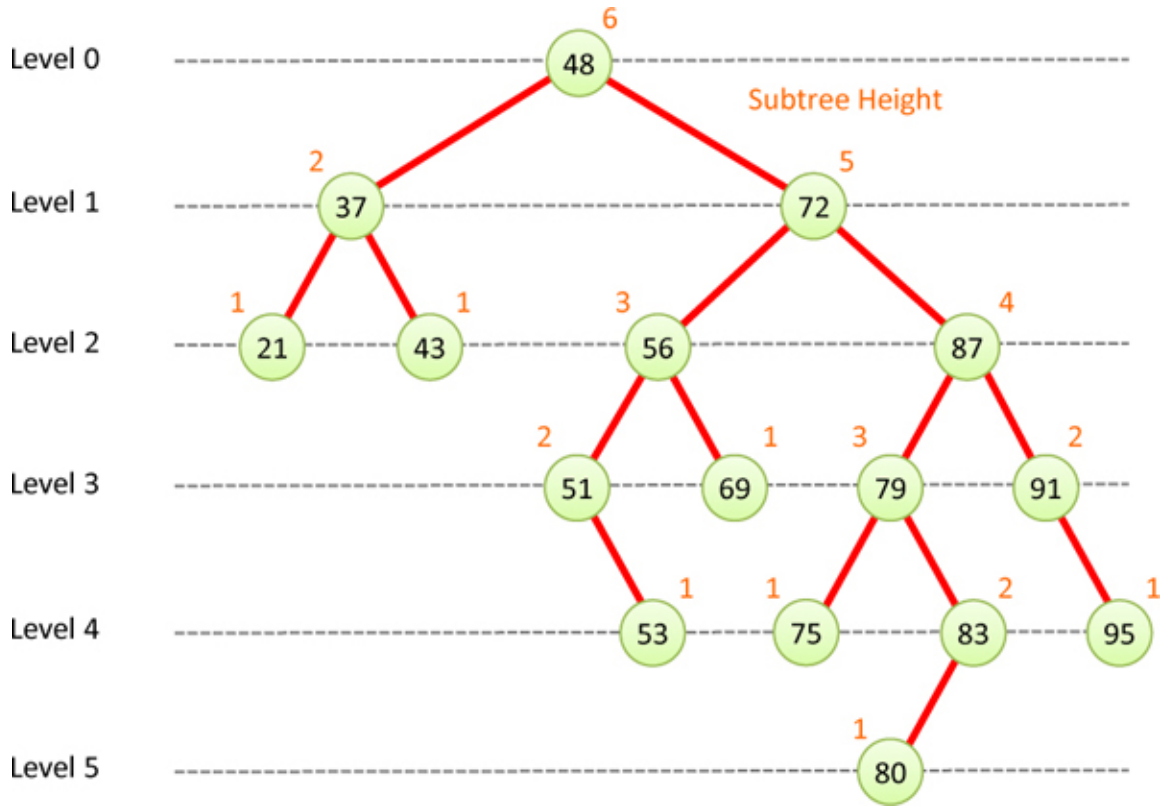

Figure 10-6 *The possible shapes for 3-node binary search trees*

Because a balanced 3-node configuration exists, you don't need to extend the definition for balanced to include anything but the middle tree in Figure 10-6. You saw that the 2-3-4 trees in Chapter 9 could be transformed into new configurations without violating any of the rules for constructing those trees. The same is true among these 3-node binary trees; they differ by simple rotations of the nodes. Ideally, the balancing algorithm could rotate the unbalanced versions into the balanced one. We come back to those rotations in a moment.

What happens when you add a fourth, fifth, sixth… node to the tree? When you add the fourth to a balanced 3-node tree, it becomes a leaf node at level 2, like the leftmost tree in Figure 10-7. Its parent has a height difference of one because it has only one child. So does the root node because it has one side with height 2 and the other with height 1. It doesn't matter which child at level

2 is the fourth node. The figure shows it being the leftmost child, but the height differences would remain for both the root and the leaf node at level 1, just with changes in sign.

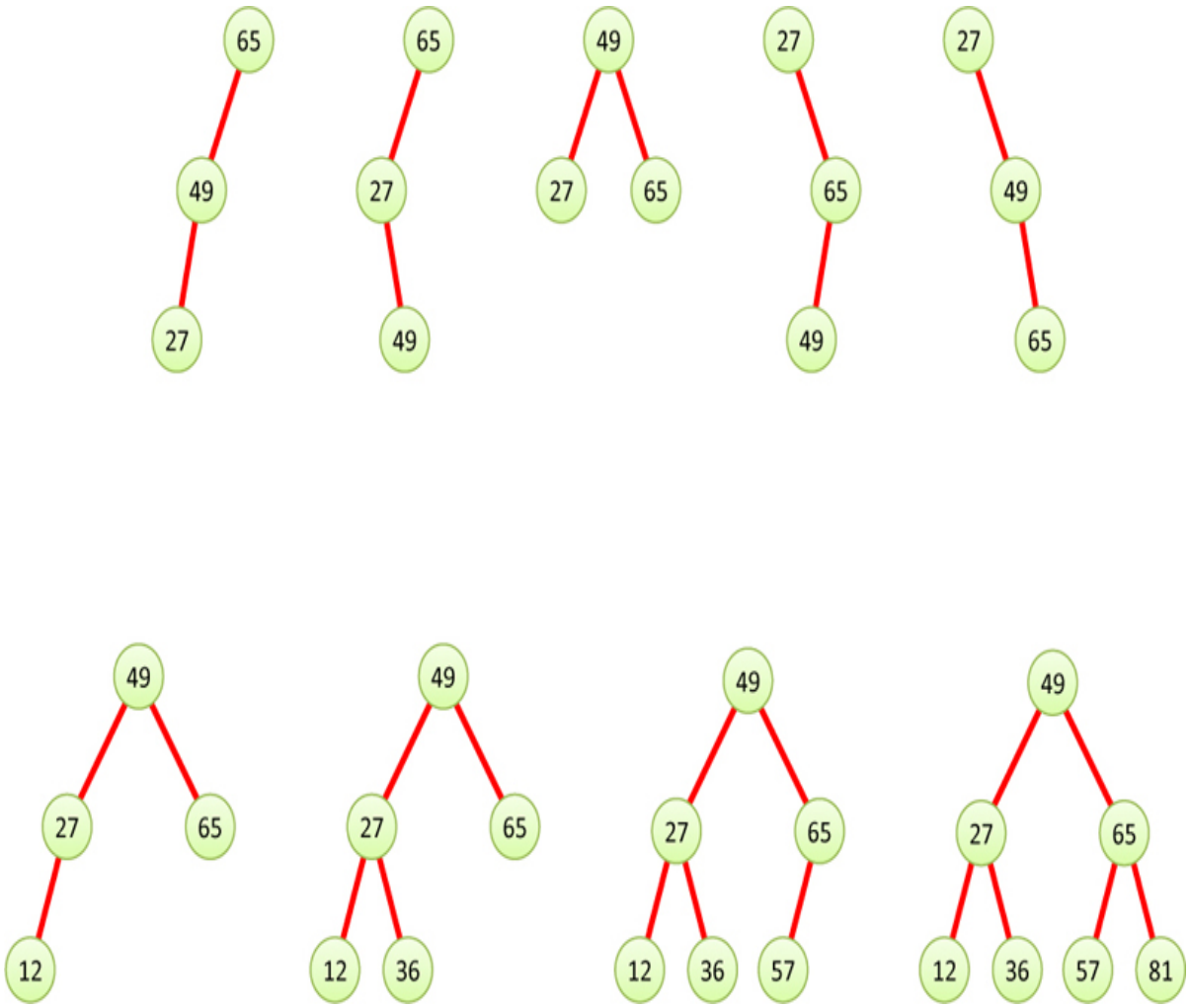

Figure 10-7 *Balanced 4-, 5-, 6-, and 7-node binary search trees*

As you add the fifth and sixth nodes, if they go at level 2 like the examples in Figure 10-7, the maximum absolute height difference of any node is one. The example shows keys that would make that happen, but even if a node were inserted at a lower level, there would be rotations that could transform the tree back into one of the shapes like those in Figure 10-7 as long as the node count was 4 through 6. Adding the seventh node means the tree can be put into the balanced shape on the right.

This same pattern repeats at every level of the tree. Newly inserted nodes go at the leaf level. That either places them in a position where no node has a height

difference of more than one, or it puts them at a level one lower than the lowest leaf, and rotations can raise them up to restore balance. This is the core idea of all self-balancing trees. We can now define balance as

A balanced, binary tree is one where all nodes have an absolute height difference of one or zero.

AVL Trees

The AVL tree is a modified binary search tree that adds a height field to each node. The height differences between the left and right children of a node can be used to measure its balance. When the absolute height difference at any subtree becomes larger than one, rotations are used to correct the imbalance. Let's look first at those rotations.

You saw the five possible configurations of 3-node binary search trees in Figure 10-6. What may not have been clear is that each of those configurations is one rotation away from another. Figure 10-8 shows those configurations in a slightly different order. To transform the tree on the far left to the next tree on its right requires a rotation around the mid-level node—the one with key 27. That rotation moves node 27 down to the left and raises node 49 to the mid-level. The next transformation in the figure shows rotating around the root, node 65, in the opposite direction. That raises node 49 to the root and lowers node 65 to the right, producing the balanced tree in the middle.

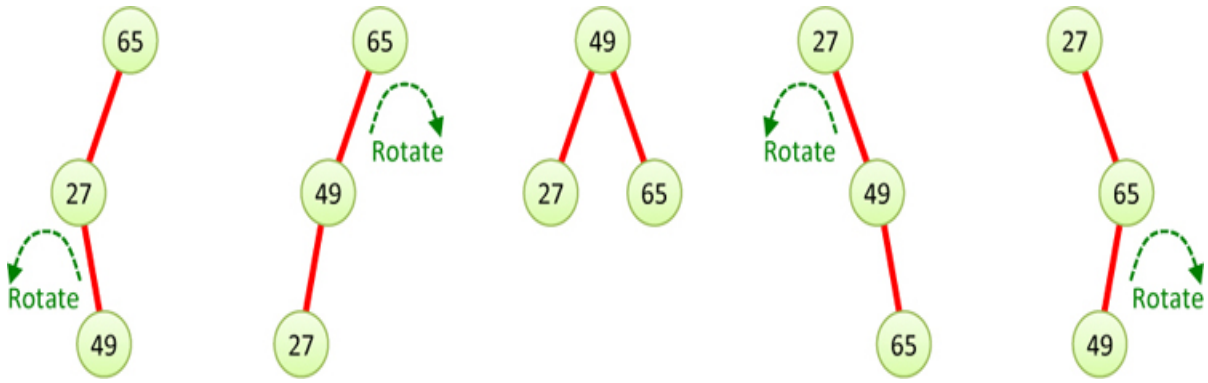

Figure 10-8 *Rotations on 3-node binary search trees*

The right side of Figure 10-8 shows the mirror image of the operations. Starting at the tree on the far right and rotating right around the mid-level node 65 produces the tree that's second from the right with node 49 at the mid-level. Rotating that tree's root to the left produces the balanced tree in the middle with node 49 at the root. You can also reverse the transform directions to go the

other way in the figure. In other words, starting from the balanced tree in the middle, rotating right around the root, produces the tree that's second from the right in the figure.

The basic idea of the AVL tree is to use the additional height field in every node to determine the balance of its parent node. When an imbalance of more than one is created by an insertion or deletion, it will be corrected by using rotations.

The AVLTree Visualization Tool

Let's explore how AVL trees work using the AVLTree Visualization tool. When you launch the tool, it starts out like the 2-3-4 Tree Visualization tool, except that the empty tree object at the top is labeled `AVLTree`. As with the other tree visualizations, only numeric keys from 0 to 99 are allowed. You can insert and search for individual keys using the Insert and Search buttons as before. There are also operation buttons for deleting, randomly filling, emptying, and traversing the nodes in order.

To start, try randomly filling the empty tree with 31 items. The tree that appears should look something like the one Figure 10-9. The heights appear above and to the right of each node in the tree.

Figure 10-9 *The AVLTree Visualization tool with a randomly filled tree*

The tree in Figure 10-9 is balanced; the height difference at all nodes is −1 , +1, or 0. That's very different from what happened when you inserted 31 randomly chosen keys into the binary search tree of Figure 10-2. Was it just luck that it

came out balanced? No, the AVL tree keeps the tree balanced with every insertion.

Try erasing the tree with the New Tree button and refilling it with 31 random nodes. The result is again a balanced tree. If you repeat the experiment many times and compare the results to the same experiment using binary search trees, you will also see that the resulting AVL trees have many more nodes, on average. Why is that?

The randomly generated AVL trees have more nodes than similarly generated binary search trees due to two factors. The first is the depth limit that the Visualization tools impose. Because insertions are not allowed at level 5 or below in either tool, some of the inserted keys are discarded. The second factor is that a balanced tree has more available nodes to fill within the depth limit. Because the AVL tree adjusts the balance on every insertion, the leaves of the tree are kept as close as possible to the root. That leaves room for later insertions. Eventually, even the AVLTree Visualization tool will run out of room at the lower levels, so it's very rare that all 31 randomly chosen values can be placed in the tree.

Inserting Items with the AVLTree Visualization Tool

Let's look at how the AVL tree maintains its balance. If you create a New Tree with the Visualization tool and then insert the keys 10 and 20 into it, you will see a tree like the one in Figure 10-10. The `flag = True` on the left indicates that the insert operation was successful (that is, it did not go past the depth limit and did not find an existing node with the same key). The 2 next to the root node indicates that it lies at height 2 in the tree.

Figure 10-10 *The first two items inserted in an AVL tree*

If you try to insert a node with key 30 in the tree, it first attempts to insert the item as the right child of node 20, following the same procedure as binary search trees do. The top panel of Figure 10-11 shows the process just after completing the insertion, before updating the height of node 20 and node 10. The procedure then returns up the path that led to the insertion point, checking the height difference of each internal node and updating its own height.

In panel 2 of Figure 10-11, the process has returned to the root node (now with the `top` arrow pointing to it) and discovered that node 10 has a height difference of −2 (a height of 0 for its empty left child and 2 for its right, node 20). Note it has not yet updated the height of node 10 at this point. Finding the absolute value of the height difference to be larger than 1, it must rebalance the tree. For that, it chooses `toRaise` node 20 as indicated by the arrow.

Figure 10-11 *Steps in the insertion of node 30 into the AVL tree*

The three nodes are rotated left, bringing node 20 to the root, as shown in the last panel of Figure 10-11. The heights of the `top` and `toRaise` nodes have been carried along in the rotation and need to be updated. Note, however, that node 30's height of 1 remains correct because height is measured relative to the leaf level.

This simplified example shows the basic operation of insertion but hides many details. In particular, this is the simplest form of rotation, where the node to raise has no children on the side between it and the top node. Let's turn now to the code and investigate all that goes on within the structure.

Python Code for the AVL Tree

The `AVLtree` class shares much of its code with that of the `BinarySearchTree` of Chapter 8 and the `Tree234` of Chapter 9. We explain the key components that differ here and leave out some of the common implementation. You can refer to the code in the previous chapters or look at the accompanying source code for the rest of the implementation.

The `AVLtree` defines its `__Node` class as private for the same reasons as in the `BinarySearchTree` and `Tree234` classes. The main difference from the `__Node` constructor for the `BinarySearchTree` is the addition of the call to the `updateHeight()` method, which creates an instance field called `height`, if it doesn't exist, and determines its value from the height of the left and right child, as shown in Listing 10-1. Because AVL trees always create new nodes as leaf nodes, the constructor doesn't take a left and right parameter for the new node. The constructor could simply set `height` to 1 without calling `updateHeight()` because the new node's left and right child are always empty. We've left that call in the code to show how the calculation is made to get the initial value of 1.

Listing 10-1 *The `__Node` Class for `AVLtrees`*

```
class AVLtree(object):

   class __Node(object):       # A node in an AVL tree
      def __init__(            # Constructor takes a key-data pair
            self,              # since every node must have 1 item
            key, data):
         self.key, self.data = key, data # Store item key & data
         self.left = self.right = None   # Empty child links
         self.updateHeight()  # Set initial height of node

      def updateHeight(self): # Update height of node from children
         self.height = max(   # Get maximum child height using 0 for
            child.height if child else 0 # empty child links
            for child in (self.left, self.right)
         ) + 1                # Add 1 for this node

      def heightDiff(self):   # Return difference in child heights
         left  = self.left.height  if self.left  else 0
```

```
        right = self.right.height if self.right else 0
        return left - right  # Return difference in heights
```

The `updateHeight()` method makes explicit the treatment of empty child links. It uses a Python list comprehension—`for child in (self.left, self.right)`—to examine both children. For links that are `None`, the child height is treated as 0, otherwise it gets the height from the `child`'s `height` attribute. The child heights are passed to the `max()` function as arguments. Finally, it adds 1 to the maximum height of the children, so that leaf nodes get a height of 1.

The decision metric, `heightDiff()`, comes next. The height of a node's right subtree is subtracted from the height of its left subtree. Because those subtrees might be empty, it substitutes a default value of zero for the child's height if the child link is `None`. Thus, a node with only one child produces a height difference that is plus or minus the height of the child.

Inserting into AVL Trees

Inserting new items into an AVL tree starts off like insertion in binary search trees; the key of the item to insert is compared with the existing keys in the tree until the leaf node is found where it should be inserted. In the `BinarySearchTree` class, insertion needed to alter only one child link, in the parent of the new node, to do its work. Because AVL trees need to check the balance after each insertion and possibly make rotations all the way up to the root, it's going to need to follow the parent links back up the tree after inserting at the leaf level. Returning up the path that was followed is easy with a recursive implementation, so that's what's shown in Listing 10-2. Keeping an explicit stack of nodes visited on the path could also do the same thing.

Listing 10-2 *The `AVLtree insert()` Method*

```
class AVLtree(object):
…
    def insert(self, key, data): # Insert an item into the AVL tree
        self.__root, flag = self.__insert( # Reset the root to be the
            self.__root, key, data) # modified tree and return the
        return flag             # the insert vs. update flag

    def __insert(self,          # Insert an item into an AVL subtree
```

```
            node,          # rooted a particular node, returning
            key, data):    # the modified node & insertion flag
      if node is None:     # For an empty subtree, return a new
         return self.__Node(key, data), True # node in the tree

      if key == node.key:  # If node already has the insert key,
         node.data = data  # then update it with the new data
         return node, False # Return the node and False for flag

      elif key < node.key: # Does the key belong in left subtree?
         node.left, flag = self.__insert( # If so, insert on left and
            node.left, key, data) # update the left link
         if node.heightDiff() > 1: # If insert made node left heavy

            if node.left.key < key: # If inside grandchild inserted,
               node.left = self.rotateLeft( # then raise grandchild
                  node.left)

            node = self.rotateRight( # Correct left heavy tree by
               node)          # rotating right around this node

      else:                  # Otherwise key belongs in right subtree
         node.right, flag = self.__insert( # Insert it on right and
            node.right, key, data) # update the right link
         if node.heightDiff() < -1: # If insert made node right heavy

            if key < node.right.key: # If inside grandchild inserted,
               node.right = self.rotateRight( # then raise grandchild
                  node.right)

            node = self.rotateLeft( # Correct right heavy tree by
               node)          # rotating left around this node

      node.updateHeight()     # Update this node's height
      return node, flag       # Return the updated node & insert flag
```

The `insert()` method is simple. It calls the recursive, private `__insert()` method starting at the root node to do the work. Note that it sets the `__root` field to the result of that call. This is how the recursive algorithm updates the tree; it calls a method operating on a subtree and then replaces the subtree with whatever modifications occurred. In this way, for instance, the empty root field gets filled with a newly created node on the first insertion. The `insert()` method returns a Boolean `flag` indicating whether a new node was inserted in the tree (as opposed to updating an existing key). The `flag` is the second value returned by `__insert()`.

Inside the `__insert()` method, the first checks are for base case conditions. If `__insert()` was called on an empty tree (or subtree), the `node` parameter is `None`, and it simply returns a new node holding the key and data of the item to be inserted. The return of that new object handles inserting the first node of the tree. It also returns the insertion flag as `True` to tell the caller that another node was added.

If the key of the node to insert matches an existing key in the tree, the `__insert()` method faces the choice of what to do with duplicate keys. Like the 2-3-4 tree, this implementation updates the existing node's `data` field with the new data to insert. That means that duplicate keys are not allowed, and the AVL tree behaves like an associative array. The caller can know this by seeing the returned insertion flag is `False`. After the `data` field is changed, the modified `node` must be returned so its parent can store the same node back in the root or other child link.

After the base cases are handled, what remains are the recursive ones. (In the AVLTree Visualization tool, one other "base" case is checked: does the insertion go past the depth limit). There are two options; the item to insert belongs in either the left or right subtree of the node. If the `key` to insert is less than this node's key, it belongs in the left side, and otherwise the right. These two choices are handled in the next two blocks of code.

For the left side, the first step is to update the `left` link with the result of recursively inserting the item in the left subtree (and record the insertion `flag` result). For programmers just getting used to recursion, that may seem like silly thing to do because it looks like setting a variable to itself. As you saw in Chapter 6, "Recursion," it can be quite powerful. You've already seen that if the `node.left` link were `None`, `__insert()` would return a new leaf node to replace it, and that's exactly what's needed. If that left link points to some subtree, you can assume it's going to return the node on top of that subtree after any insertions and rotations that might happen inside it to maintain the subtree's balance. Having made that assumption, all that's left to do is complete the work after the balanced subtree is returned. Of course, you have to return the revised subtree from this call to `__insert()` to ensure the assumption remains accurate.

What work must be done after the left subtree is updated? Well, it's possible that the insertion on the left caused the balance of this subtree to become **left heavy**; that is, the left subtree has a height that is now greater than that of the right subtree by more than one. It cannot have made this node **right heavy**

because the insertion was made in the left subtree and you started off with a balanced subtree (because AVL trees are designed to always maintain self-balance). Another way of saying that is the height difference of `node` must have been either −1 , 0, or +1 when the `__insert()` method was called.

The `__insert()` method checks the node's balance by seeing whether `node.heightDiff()` now exceeds 1. As shown in Listing 10-1, `heightDiff()` computes the difference of the (newly modified) height of `node`'s left subtree with that of `node`'s right subtree. If the left node's height was updated properly in the recursive call, this will work. If you look ahead in the code, that update happens right before the `__insert()` method returns the modified node.

When `node` is left heavy after the insertion in its left child, the `__insert()` method needs to correct it by performing one or two rotations. To decide which ones are needed, it looks to see where the key was inserted relative to the node. Back in Figure 10-8, you saw how to rotate 3-node trees until they were balanced. The same logic applies here, under the assumption that the item just inserted was the third node.

When the insertion goes in the left child of `node`, then the shape of the tree is either the leftmost tree in Figure 10-8 or the second to leftmost. The insertion must have produced a grandchild; otherwise, it couldn't have made `node` left heavy. The question then becomes, Was the item inserted in the *inside* or *outside grandchild* of the `node`? The leftmost tree of Figure 10-8 shows an insertion at the *inside grandchild* of the top node. That requires two rotations to get the balanced form in the middle tree of Figure 10-8. If the insertion was to the *outside grandchild*, the situation requires only one rotation.

The `__insert()` code in Listing 10-2 checks whether the `key` that was inserted is larger than `node`'s left child key. If it is, then the insertion was an inside grandchild because the insertion path went left and then right on the way down. To correct the imbalance, it performs a left rotation around `node`'s left child. Otherwise, it performs the only one rotation—a right rotation around the top of the subtree, `node`. That's what's needed if the insertion went to the outside grandchild, going left and left again on the way down. The right rotation corrects the left heaviness of the tree. The rotation operations are performed by methods we describe in a moment. They use the same style as the recursive `__insert()` method and set the node at the top of the rotation to whatever node is moved up.

The preceding steps handle the rotations for insertions in `node`'s left subtree. The next `else` clause handles the rotations for insertions in `node`'s right subtree. These are the mirror image of the steps in the left-hand rotations, swapping right and left and changing the sign of the height difference. The inside grandchild is the one following a right and then a left child link, and the outside follows a right-right link.

After either kind of subtree insertion, the `__insert()` method calls `updateHeight()` on the `node`, followed by returning the modified node and the insertion `flag` from the recursive call. The height could be different than the node's height on entry to the `__insert()` method because there is one more item in a subtree below. Updating the height at this point ensures that the node returned to the caller has proper information about its position relative to the leaves of the subtree.

Rotations are performed by changing the links between the nodes in the subtree. The `top` of the subtree is sometimes called the center of rotation or the pivot (although that latter term might be confused with the pivot used in quicksort). For `rotateRight()`, the `top`'s left child is raised up. The code of Listing 10-3 shows `top.left` being stored in the variable `toRaise`.

Listing 10-3 *The `rotateRight()` and `rotateLeft()` Methods of `AVLtree`*

```
class AVLtree(object):
…
   def rotateRight(self, top): # Rotate a subtree to the right
      toRaise = top.left       # The node to raise is top's left child
      top.left = toRaise.right # The raised node's right crosses over
      toRaise.right = top      # to be the left subtree under the old
      top.updateHeight()       # top. Then the heights must be updated
      toRaise.updateHeight()
      return toRaise           # Return raised node to update parent

   def rotateLeft(self, top): # Rotate a subtree to the left
      toRaise = top.right      # The node to raise is top's right child
      top.right = toRaise.left # The raised node's left crosses over
      toRaise.left = top       # to be the right subtree under the old
      top.updateHeight()       # top. Then the heights must be updated
      toRaise.updateHeight()
      return toRaise           # Return raised node to update parent
```

The next step is perhaps the most confusing. This is where the node being raised has its right child moved over to become `top`'s left child. The moving child is the **crossover subtree**. The right child is empty in the 3-node tree example of Figure 10-8, so it appears to be just a way to initialize the left child of `top` to be empty in its new position. We discuss what happens when the crossover contains a subtree in a moment.

After the crossover link is moved, `top` becomes the right child of the node `toRaise`, completing the restructuring of the nodes. The heights of both changed nodes need to be updated because their relative positions changed. You change the height of `top` first because it moved lower in the tree, and the height of `toRaise` now depends on it. When both heights are updated, then the new top of the subtree, `toRaise`, is returned to be stored in the parent's link to the rotated subtree.

The code for `rotateLeft()` in Listing 10-3 is another mirror image of the code for `rotateRight()`, swapping the roles of left and right.

The Crossover Subtree in Rotations

What happens during rotations of more complex trees when rotations bubble up the tree after inserts at lower levels? There could be child links all around the nodes being rotated. Where do they go? To answer that question, let's look at a right rotation in the middle of a tree.

When you rotate right about some node, you are trying to raise its left child up. Let's call the node at the top, T, and the node to raise, R. In general, three subtrees appear below T and R, as shown in Figure 10-12. R's left child has all the nodes with keys less than R's key, and R's right child has all the keys greater than R's key. More specifically, it has all the keys between R's key and T's key. The reason is that you had to follow the left child link of T when inserting those keys. The third subtree that's involved is T's right child, which has nodes with keys greater than T's key.

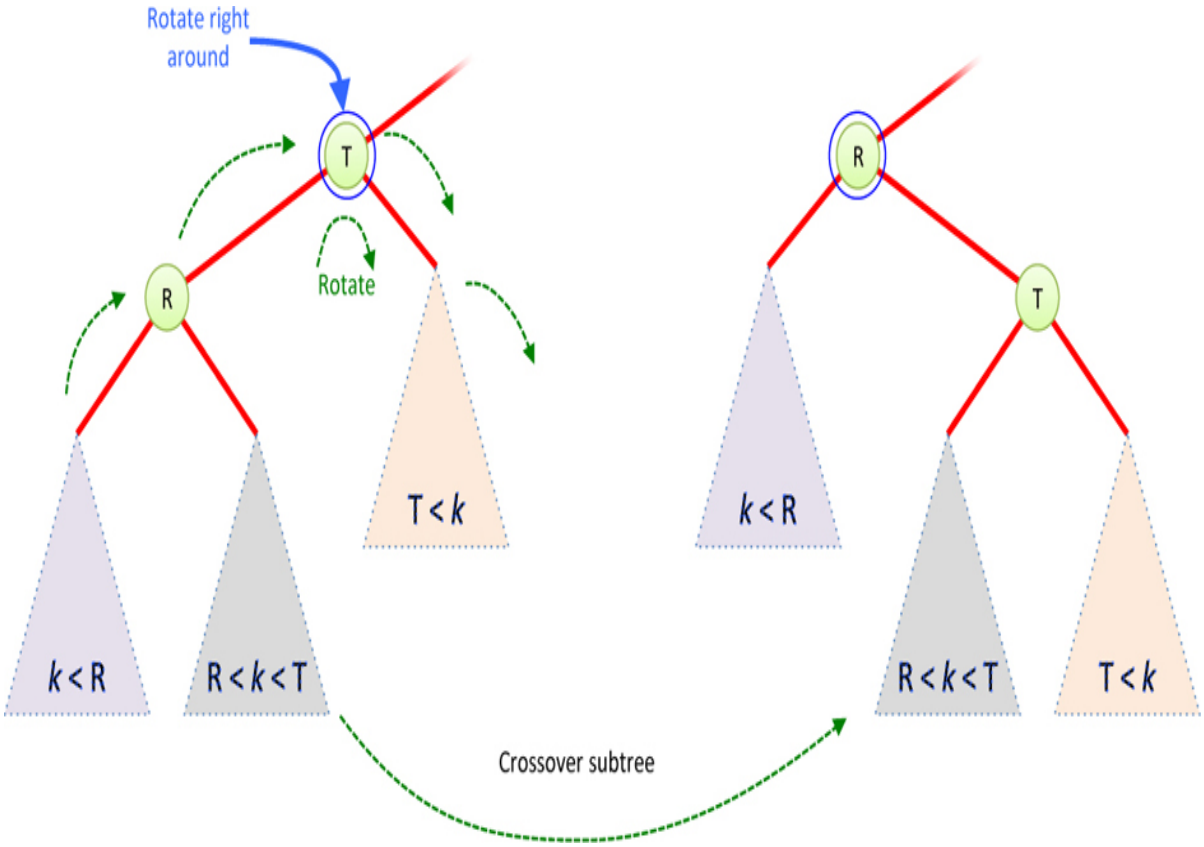

Figure 10-12 *Right rotation and the crossover subtree*

The left side of Figure 10-12 shows the relationships of T, R, and its three subtrees before the rotation. T sits at the top with R as its left child. T may or may not have a parent node, indicated by the fading line leading upward. Hanging below T and R are the three subtrees, containing the keys less than R (k < R), the keys between R and T (R < k < T), and the keys greater than T (T < k). The green arrows show the planned movement direction of the nodes. R moves up and to the right, while T moves down and to the right.

The outer two subtrees are obviously still in the correct position after the rotation produces the configuration on the right of the figure. What about the crossover subtree? It holds the keys higher than R's and less than T's key. Because it now is the left child of T, any search that gets to T will correctly choose the left link to find the keys it contains. Because T became the right child of R, any search for a key larger than R's will correctly choose its right link to T, and hence reach the crossover subtree.

The right rotation preserves the items in the correct subtrees relative to the keys being rotated. Also, the heights of the subtrees do not change during the

rotation, so the call to `updateHeight()` in `rotateRight()` only needs to check the height of the topmost node of each subtree to correctly update the heights of T and R (`top` and `toRaise`). The `rotateLeft()` method performs the mirror image set of operations, preserving the subtree relationships.

The visualization tool animates the movements of the various subtrees during rotations. Figure 10-11 showed the simple example of a left rotation with an empty crossover subtree. Try creating situations where subtrees cross over from left to right (or vice versa). A simple example happens when you insert the sequence of keys: 10, 20, 30, 40, 50, 60. After node 60 is inserted, use the stepping capabilities to watch the updates to `top` and `toRaise` in the `rotateLeft` method when it is called to rotate the root node. This shows how the crossover subtree (rooted at node 30) gets put in place. Another example that shows the double rotation of an inside grandchild is to add keys 25 and 27 to the six just inserted. Then add keys 29, 28, and 22 to see a 3-node crossover subtree movement. Watch these animations several times until you can easily anticipate what will happen before the nodes are moved.

Deleting from AVL Trees

Deletion from an AVL tree is somewhat more complex and follows the strategy used for the binary search trees described in Chapter 8. Deleting leaf nodes is easy, although you must check the tree's balance afterward. Deleting an internal node is not much harder if it has a single child link; the child replaces the node in the tree. Only deletion of internal nodes with two child links poses significant challenges. Even then, you know that you can replace the item stored at that node with that of its successor, which always lies at one of the simpler deletion positions.

As was the case for insertions, each deletion in a subtree could cause the height to change enough that the tree is no longer balanced. Correcting the imbalance will involve rotations—in fact, the same kinds of rotations used for insertions. The implementation can use the same kind of recursive method to find the node to delete and then to hunt for the successor, updating the child links and heights after each recursive call completes.

The main `delete()` method shown in Listing 10-4 serves the same role as the `insert()` method; it calls the private recursive `__delete()` method on the root node, updates the root with the returned value, and returns a `flag` indicating whether the goal node was found and deleted.

Listing 10-4 *The* `delete()` *method of* `AVLtree`

```
class AVLtree(object):
…
   def delete(self, goal):     # Delete a node whose key matches goal
      self.__root, flag = self.__delete( # Delete starting at root and
         self.__root, goal)    # update root link
      return flag              # Return flag indicating goal node found

   def __delete(self,          # Delete matching goal key from subtree
                node, goal):   # rooted at node. Return modified node
      if node is None:         # If subtree is empty,
         return None, False    # then no matching goal key

      if goal < node.key:      # Is node to delete in left subtree?
         node.left, flag = self.__delete( # If so, delete from left
            node.left, goal)   # update the left link and store flag
         node = self.__balanceLeft(node) # Correct any imbalance

      elif goal > node.key:    # Is node to delete in right subtree?
         node.right, flag = self.__delete( # If so, delete from right
            node.right, goal) # update the right link and store flag
         node = self.__balanceRight(node) # Correct any imbalance

      # Else node's key matches goal, so determine deletion case
      elif node.left is None: # If no left child, return right child
         return node.right, True # as remainder, flagging deletion
      elif node.right is None: # If no right child, return left child
         return node.left, True # as remainder, flagging deletion
      # Deleted node has two children so find successor in right
      else:                    # subtree and replace this item
         node.key, node.data, node.right= self.__deleteMin(node.right)
         node = self.__balanceRight(node) # Correct any imbalance
         flag = True           # The goal was found and deleted

      node.updateHeight()      # Update height of node after deletion
      return node, flag        # Return modified node and delete flag
```

The `__delete()` method works on a subtree rooted at a `node` in the tree. First it checks the base cases. If `node` is `None`, the subtree is empty, so nothing can be deleted. The empty subtree is returned along with `False` to indicate no deletion occurred.

For subtrees with some items, the next checks determine where the `goal` lies relative to the root of the subtree, `node`. If the `goal` key is less than that of `node`, the item to delete must be in its left subtree. It updates that left subtree with the result of the recursive call to `__delete()` on `node.left`. Then it rebalances the node knowing that the left side has been reduced by calling `balanceLeft()`. Similarly, if the `goal` key is greater than that of `node`, the item to delete must be in its right subtree. It performs the delete on the right and rebalances accordingly. For both branches, it stores the `flag` indicating whether a deletion actually happened.

If neither of those `if` statement conditions applied, then `__delete()` knows the `goal` key matches the `node`'s key. Now it can look to see what deletion case it has. If the `node` has no left child, then deleting this node is simply a matter of promoting its right child to replace it in the tree. If both the left and right child links are empty, then `node` is a leaf, and returning `node.right`, which is `None`, will prune it from its parent. The next `elif` statement checks if the right child link is empty. If so, it simply promotes the left child to replace the node.

After all the simple deletion cases have been checked, you know you are deleting an item from a node that has two subtrees. You must find the successor for this node, store its item in this node's key and data fields, delete the successor from the tree, correct any balance issues that causes, update this node's height, and return the modified node. That's quite a bit of work to do, and it's broken out into a couple of helper methods.

The `__delete()` method calls the `__deleteMin()` method to delete the item with the minimum key in the node's right subtree. That minimum is the successor to this node, as you learned in Chapter 8. Then it calls the `__balanceRight()` method to correct any imbalance caused by removing the successor from the right subtree. It also sets `flag` to `True` indicating a deletion happened.

After any of the different types of deletion are performed, the node's height could have changed, so it calls `node.updateHeight()`. Finally, it returns the modified `node` and the deletion `flag`. We look at each of those new methods to see how they do their work.

The `__deleteMin()` method returns three values using a Python tuple. The first two are the key and data of the successor node. They must replace the item being deleted in the node that `__delete()` found. The third returned value is the modified node for the right child of the node that `__delete()` found. It's

possible that the right child is a leaf node (making it the successor), and that would mean returning `None` for the third value.

The `__deleteMin()` method shown in Listing 10-5 operates like `__delete()` in that it recursively descends the subtree to find and delete an item. The item to find is not known by a specific key; it's the item with the smallest key, so the recursion always follows the left link until a node with an empty left child is found. That's the base case, which is tested first. After that minimum node is found, its key and data can be returned as the successor. It returns the right child of the successor, which might be empty, as the subtree to replace the minimum node. After that's returned to its caller, the minimum node has been removed from the tree. (There's no need for a recursive call to `__delete()` here like there was for deleting the successor in binary search trees because the assignment to `node.left` from the result of the recursive call eliminates the `node` from the tree.)

Listing 10-5 *The `__deleteMin()` and Rebalance Methods of `AVLtree`*

```
class AVLtree(object):
…
   def __deleteMin(            # Find minimum node of subtree, delete
         self,                 # it, return minimum key, data pair and
         node):                # updated link to parent
      if node.left is None:    # If left child link is empty, then
         return (node.key, node.data, # this node is minimum and its
                 node.right)   # right subtree, if any, replaces it
      key, data, node.left = self.__deleteMin( # Else, delete minimum
         node.left)            # from left subtree
      node = self.__balanceLeft(node) # Correct any imbalance
      node.updateHeight()      # Update height of node
      return (key, data, node)

   def __balanceLeft(self, node): # Rebalance after left deletion
      if node.heightDiff() < -1: # If node is right heavy, then
         if node.right.heightDiff() > 0: # If the right child is left
            node.right = self.rotateRight( # heavy, then rotate
               node.right)     # it to the right first

         node = self.rotateLeft( # Correct right heavy tree by
            node)              # rotating left around this node
      return node              # Return top node
```

```
def __balanceRight(self, node): # Rebalance after right deletion
   if node.heightDiff() > 1: # If node is left heavy, then
      if node.left.heightDiff() < 0: # If the left child is right
         node.left = self.rotateLeft( # heavy, then rotate
            node.left)     # it to the left first

      node = self.rotateRight( # Correct left heavy tree by
         node)             # rotating right around this node
   return node             # Return top node
```

The rest of the `__deleteMin()` method handles the recursive work of descending the left subtree, storing the successor key and data that were found, adjusting any balance problems the lower deletion in the left subtree causes, and updating the height of `node` after the deletion and possible rotations. These actions are like those for insertions, although the call to the `__balanceLeft()` method is different. Before we describe how that works, let's first look carefully at which subtrees get rebalanced by `__deleteMin()`.

Because the call to `__balanceLeft()` occurs immediately after every recursive descent down the left child links, it is applied to every node visited that has a left child. The successor node, by definition, does not have a left child. Could not calling `__balanceLeft()` on the successor cause a problem?

To answer that question, think about two possibilities for the successor node: a leaf node, or a node with a right subtree and no left subtree. In the first case, the leaf node is being deleted, so there really isn't anything that could be balanced. In the second case, the successor is replaced by its right subtree. The right subtree, call it S, must have been balanced before the call to `__delete()` because AVL trees preserve balance for insertions and deletions. The balance measure of S depends solely on the height of S's subtrees, which don't depend on their position in the overall tree. So, you can safely skip rebalancing the successor node after updating it with an already-balanced subtree rooted at S. The successor's parent node, however, may need rebalancing because S will have a height that is one less than the successor that is being moved. The caller will take care of balancing the parent of the successor.

The `__balanceLeft()` method might not be called in one other case. That happens when the first call to `__deleteMin()` lands on a node with no left child. The `node` in that case is the right child of the node that matched the goal key (which will be updated with the successor key shortly). With `__deleteMin()` immediately returning its `node`'s item and right subtree in its base case, no call to the `__balanceLeft()` method occurs. Using the same

reasoning as earlier, however, you know that the right subtree being returned must have been balanced before the call to `__delete()`. The already-balanced subtree is being promoted up to replace the `node` in the call to `__deleteMin()` and doesn't need rebalancing. When `__deleteMin()` returns, the `__delete()` method calls `__balanceRight()` to rebalance the shortened right subtree.

Rebalancing all the nodes that need it is a little tricky. The recursive structure in the delete routines follows every link and applies the `__balanceLeft()` or `__balanceRight()` method at every level. That should give you confidence that you can't have missed any nodes along the way.

Now it's time to look at the `__balanceLeft()` and `__balanceRight()` methods themselves in Listing 10-5. They use the `__heightDiff()` method to determine if the subtree rooted at the node is right heavy or left heavy, respectively. If the subtree is not heavy on one side, it does nothing and returns the unmodified `node` (and hence its subtree).

In `__balanceRight()` when the subtree to rebalance is left heavy, it's almost the same situation as when the `__insert()` method had to deal with adding a new item in the left subtree. This time, however, you don't have an insert key to know where the extra height was added (because a node was deleted from the right subtree). Instead, you need to look at the balance—the height difference—of the left subtree, the one that needs to be raised to restore balance. If that left subtree is perfectly balanced or has a left height one greater than its right (a height difference of 0 or +1), then you will need only one right rotation to bring it up to the top. Note that the left subtree's height difference can take on only three possible values: −1, 0, or +1. Any other value would mean the subtree was unbalanced, and you've balanced every subtree before passing it up the recursive chain.

What happens when the left subtree of the node to rebalance in `__balanceRight()` has a height difference of −1? That difference indicates that its right subtree's height is one more than its left. That situation is shown on the left of Figure 10-13. The `__balanceRight()` method was called on node T and found that it is left heavy due to the deletion in its right subtree. That means it found a height difference of +2 at T. Furthermore, T's left subtree, rooted at node R, has a height difference of −1. That means the right subtree has a height that is one more than that of the left. In the figure they are shown with heights of H and H + 1. It's that right side with height H + 1 that is going to cross over when the `__balanceRight()` method rotates T to the right.

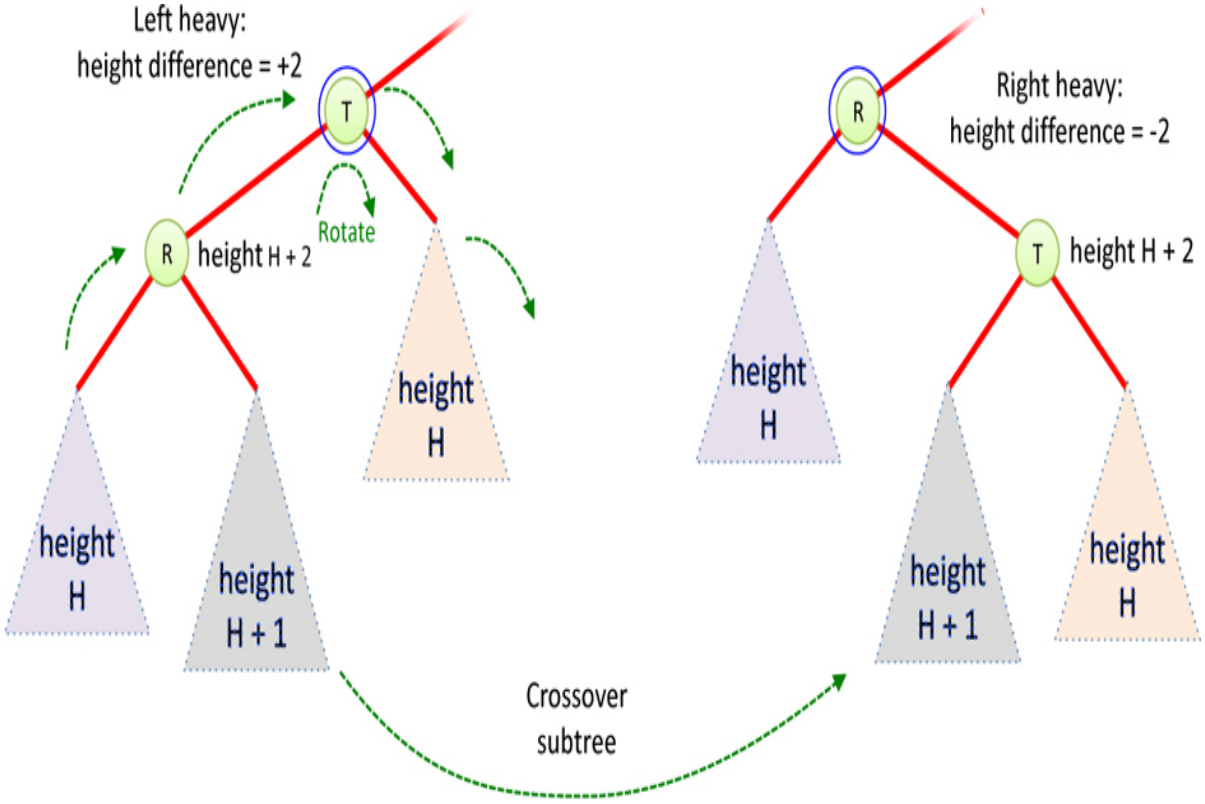

Figure 10-13 *Balancing a left-heavy subtree*

In this situation, the right subtree of T must have a height of H. You know that because the height difference at T is exactly +2, and R's height is H + 2. If you execute a right rotation around T, you will get the situation on the right of Figure 10-13. The right subtree of R with height H + 1 crosses over to become the left subtree of T. The height of T would then be H + 2 because it is one more than the height of its maximum child. Comparing the height difference at R, now the top node after the right rotation, you find −2, meaning it is a right-heavy tree. That's a problem, but you can solve it with another rotation.

The solution is to use the same operation used when handling the inside grandchild after an insertion on the left subtree (see Figure 10-8). If you first use a left rotation on the left subtree, R, that will reduce the height of the crossover subtree to H. By raising the inside grandchild of T up one (rotating left around R), you have evened up the left side prior to raising the left child of T up to be the new top of the tree (in the rotation right around T).

Thus, the `__balanceRight()` method performs one or two rotations to correct an unbalanced subtree. It checks the height differences at the root of the subtree and at its left child to determine which rotations are needed. The

`__balanceLeft()` method performs the mirror image operations to correct a left deletion that caused a right-heavy subtree. Note that it does not need to recursively descend into the subtrees. The reason is that you've ensured they all have an absolute height difference of one or less in previous rebalancing operations.

The visualization tool shows how all these methods operate. Try deleting nodes from a tree: first a few leaves, then some internal nodes with one and two children to see all the cases. You can click a node with your pointer device to enter its key in the text entry box or type the key number before selecting the Delete button. After several deletions on one side, the AVL tree will eventually perform a rotation using the `__balanceLeft()` or `__balanceRight()` method. Use the stepping controls to carefully follow any of the calls that cause confusion.

We've now covered insertion and deletion in AVL trees. The remaining methods of AVL trees such as `search()` and `traverse()` are the same as for binary search trees, as you can see if you look carefully at the visualization tool's code. The added height field of nodes might not be exposed to callers because it is only used to maintain the balance of the tree, but it can be a useful metric for some applications.

The Efficiency of AVL Trees

Because AVL trees are similar to binary search trees, their efficiency is similar. There is the added storage cost of keeping the height field with each node and the added time it takes to update that field and rebalance subtrees during insertions and deletions. That extra work produces balanced, binary trees, which you know have a search time of O(log N). Best of all, the search time doesn't degrade to O(N) for degenerate cases.

How much do you have to pay to get the benefits of balance? On the insertion, there are the calls to `updateHeight()` at every level of recursion. There are $\log_2(N + 1)$ levels in the tree and the same number of recursion levels. There are also calls to `heightDiff()` at every level to check the tree's balance. Some of those lead to more key comparisons and rotations. In the worst case, every level would call `heightDiff()` once and make two rotations (which call `updateHeight()` twice each). That's quite a bit, but it's still a fixed amount of work that doesn't depend on the number of nodes in the tree. All the added manipulations increase the amount of work done per recursive level by a

constant amount. In Big O notation, you can ignore the constant amount and conclude that insertion is O(log N) because the only thing that grows with N is the number of levels.

The analysis of deletion follows that of insertion. There are still $\log_2(N + 1)$ levels to visit, and each one involves calls to `updateHeight()` and `heightDiff()` inside of `__balanceLeft()` or `__balanceRight()`, and one or two rotations when imbalances are found. Those calls all add to the constant that multiplies the number of levels, and that constant doesn't change as the number of items grows larger. That leaves deletion as an O(log N) operation.

Thinking about the amount of memory used, it is O(N) because you need a fixed amount of memory for every node, and there is exactly one node for every item stored. The constant that multiplies the number of items, however, has grown because you added the height field to the tree. Somewhat harder to see is that the height field needs to be big enough to accurately count the height of each subtree. If you used a single byte for the height field, then it could only count up to a height of 255. That limitation could become a problem for a huge number of items. A full machine word could be used to store the height, which would be 64 (or possibly 32) bits, allowing for extremely deep trees.

There's also the O(log N) memory that will be used for the recursive stack because there is at least one call per level of the tree. There are, of course, many other procedure calls, but the total number that are active at any one time will be proportional to the number of levels. Because the O(log N) memory will be much smaller than the O(N) memory needed to store the nodes, it is usually ignored.

Red-Black Trees

Are there other ways to balance binary trees? Yes, of course there are, but are there any that are just as efficient but don't add as much extra work as the AVL tree does? That's a bigger challenge, but a structure called the **red-black tree** provides an interesting answer. Instead of relying on a measure of a node's height, all the red-black tree requires is 1 bit more of information about each node. The bit encodes whether the node is labeled as red or black. How can a single bit help balance the tree? Let's find out.

Red-black trees are not trivial to understand. Because of this, the actual code is more lengthy and complex than you might expect. It's therefore hard to learn

about the algorithm by examining code. You've seen many of the implementations of key operations like rotations and determining inside and outside grandchildren in the binary search tree and AVL tree. So, we don't describe the red-black tree source code in this text. Instead, we discuss the structure and algorithms, and help you understand the operations using another visualization tool.

Conceptual

For a conceptual understanding of red-black trees, we are aided by the RedBlackTree Visualization tool. We describe how you can work in partnership with the tool to insert new nodes into a tree. Including a human into the insertion routine certainly slows it down but also makes it easier for the human to understand how the process works.

Searching works the same way in a red-black tree as it does in an AVL or ordinary binary search tree. As you might expect, insertion and deletion are where the differences emerge. Accordingly, in this chapter we concentrate on the insertion process.

Top-Down Insertion

Red-black trees use *top-down* insertion. This means that some structural changes may be made to the tree as the search routine descends the tree looking for the place to insert the node. This approach was used in the 2-3-4 trees in Chapter 9 when full nodes were split as the algorithm searched for an insertion point.

Bottom-Up Insertion

Another approach is *bottom-up* insertion. This type involves finding the place to insert the node and then working back up through the tree making structural changes. Bottom-up insertion is less efficient because two passes must be made through the levels of the tree. The 2-3 tree in Chapter 9 used a bottom-up approach, splitting full leaf nodes and promoting the overflow up toward the top (root). The AVL tree also rebalances the tree in a bottom-up fashion. In a red-black tree, balance is maintained during insertion and deletion, as was done for all the self-balancing trees. When an item is inserted, the insertion routine

checks that certain characteristics of the tree are not violated. If they are, it takes corrective action, restructuring the tree as necessary. Because the routine maintains these characteristics, the tree is kept balanced.

Red-Black Tree Characteristics

In a red-black tree, every node is marked as either black or red. These are arbitrary colors; blue and yellow would do just as well. In fact, the whole concept of saying that nodes have colors is somewhat arbitrary. Some other analogy could have been used instead. You could say that every node is either heavy or light, or yin or yang. Colors, however, are convenient labels and help programmers use a consistent vocabulary. A Boolean data field, such as `isRed`, is added to the node class to embody this color information.

In the RedBlackTree Visualization tool, the red-black characteristic of a node is shown by its border color. The center color, as it was in all the tree visualization tools, is simply an indication of the data field of the node. When we speak of a node's color in this chapter, we are almost always referring to the solid red or black border color shown in the figures.

Red-Black Rules

When inserting (or deleting) a new node, you check for certain conditions, which are called the **red-black rules**. If they're all followed, the tree is called **red-black correct**, and it will be balanced.

> Rule 1. Every node is either red or black.
>
> Rule 2. The root is always black.
>
> Rule 3. If a node is red, its child nodes must be black (although the converse isn't necessarily true).
>
> Rule 4. Every path from the root to a leaf, or to a null child, must contain the same number of black nodes.

The "null child" referred to in Rule 4 is a place where a child could be attached to a nonleaf node. In other words, it's the potential left child of a node with only a right child or the potential right child of a node with only a left child. This description will make more sense as we go along.

The number of black nodes on a path from the root to a leaf is called the **black height**. Another way to state Rule 4 is that the black height must be the same

for all paths from the root to a leaf or a null child. Note that the black height is defined for a path, while the height metric used for AVL trees is for a whole subtree (and doesn't care about node colors).

These rules might seem completely mysterious. It's not obvious how they will lead to a balanced tree, but they do; some very clever people invented them. Copy them down somewhere and keep them handy. You'll need to refer to them often as you learn about red-black trees.

Aside from the basic rules, red-black trees are as flexible as binary search trees. They allow any kind of key, as long as they can be compared to determine an ordering. They don't allow duplicate keys and act as an associative key store. People familiar with the red and black coloring of roulette wheels might infer those keys must be odd or even numbered in some way, but that is wrong. You won't always see alternating colors along every path; rule 3 only prevents red nodes from having child nodes that are also red. The black nodes can have black or red child nodes.

Fixing Rule Violations

Suppose you see (or are told by the visualization tool which we introduce shortly) that the color rules are violated. How can you fix your tree so that it complies? There are two, and only two, possible actions you can take:

- You can change the colors of nodes.
- You can perform rotations.

In the tool, changing the color of a node means changing its red-black border color (not the center color). Changing the color means setting the Boolean flag to a different value for one or more nodes. Because it's a Boolean, you can call this "flipping" the color between one of the two values, akin to flipping a coin. Rotations, as you have seen, rearrange some nodes in a way that, one hopes, leaves the tree more balanced. They always preserve the search relationships of the keys. The colors of the nodes stay with them as they rotate, and that "stickiness" can cause (or fix) rule violations.

At this point such concepts probably seem very abstract, so let's become familiar with the RedBlackTree Visualization tool, which can help to clarify things.

Using the Red-Black Tree Visualization Tool

To launch the Visualization tool, follow the instructions in Appendix A, "Running the Visualizations," and either run the `RedBlackTree.py` program or select it from the menu of visualizations. Figure 10-14 shows what the RedBlackTree Visualization tool looks like when it starts with two nodes inserted.

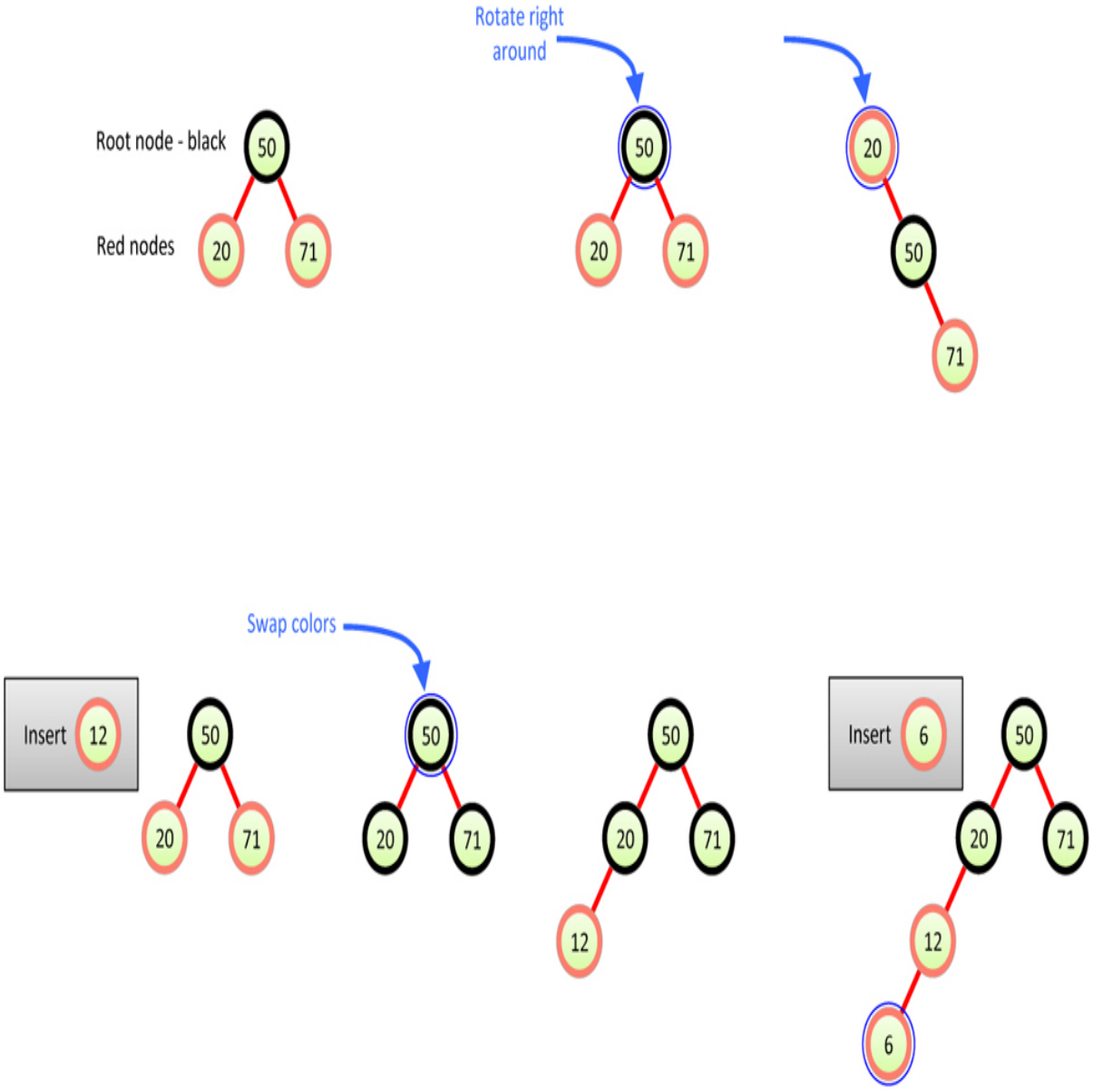

Figure 10-14 *The RedBlackTree Visualization tool*

Like the other trees you've studied, a `RedBlackTree` object has a single pointer to the root node of the tree. In the figure, node 77 is the root and is colored black. It has one child on the right, node 94, that is colored red.

In the upper left, three messages about the red-black rules appear (the first rule—that all nodes are colored red or black—is always enforced by the tool). Rule 2 is either true or false, depending on the color of the root. For Rule 3, the message counts the number of red nodes linked to other red nodes. For Rule 4, the message shows all the different black heights that can be found in the tree. In other words, it computes the black height for every leaf and null child in the tree and shows the set of unique heights inside of curly braces. In the case of Figure 10-14, node 94 is a leaf with a black height of 1, and the root node has a null left child that also has a black height or 1.

The red-black rules are all satisfied for this tree, so the messages are shown in green, and the last message says `" RED-BLACK CORRECT!"` That message disappears when any of the rules are violated.

Flipping a Node's Color

You can change a node's color by either clicking it with a pointer device or entering its key in the text entry box and selecting the Flip Color button. Try changing the color of node 94 to black. The ring color changes, and the messages change to show that one of the rules is violated, as shown in Figure 10-15. With exactly two black nodes in the tree, there are now two different black heights: 1 for the null left child of the root and 2 for leaf node 94. Flip node 94 back to red, and the rules are satisfied again.

Figure 10-15 *A tree with exactly two nodes, both black*

One side effect of clicking a node to flip its color is that the node's key is entered into the text entry box for use in subsequent operations. This is a little different from other operations you've seen where the text entry box is cleared after an operation completes.

With node 94 red again, flip the color of root node 77 to red. Now two of the rules are violated, as shown in Figure 10-16. The root node is not black, violating Rule 2, and the one parent-child link is between two red nodes, violating Rule 3. The link is highlighted in red to indicate the problem. The two red nodes, however, satisfy Rule 4 because all the black heights are now 0.

Figure 10-16 *A tree with exactly two nodes, both red*

Rotating Nodes

As you saw with AVL trees (and discussed for 2-3-4 trees), rotating around a node can change the balance of a tree. In the RedBlackTree Visualization tool, you select a top node of the subtree to rotate and then select either the Rotate Left or Rotate Right button to perform the rotation. You can select the top node by either typing its key in the text entry box or by clicking it with the pointer device to copy the key. The single click also changes its color, and if that is not desirable, you can click it a second time to revert the color.

Double-clicking a node also rotates that node to its right. The messages about the rules temporarily disappear. After the nodes are repositioned, the red-black rules are rechecked. If you hold down the Shift key or use the second mouse button when double-clicking, the rotation goes to the left. If you ask to rotate a

node without a subtree to raise into its place, a message appears below the operations area explaining the problem.

The Insert Button

The Insert button causes a new node to be created, with the key that was typed into the text entry box (assuming there is space for it in the Visualization tool tree). Although the code isn't shown, it performs the familiar binary search tree algorithm to find where the key should be located. If the key already exists, it updates the data for that key by giving the data circle a new color. If the key doesn't exist but would go in a node at level 0 through 4, it inserts the new node with the key and data. Attempts to insert keys at level 5 or below fail with an error message. (Note also that rotations that move nodes to level 5 will delete those nodes.)

We discuss the choice of the red-black ring color of the new node in the next section. The color, of course, affects the red-black rules, so changes may be needed to keep the tree balanced.

The Search Button

The Search button acts like the one you've seen in the other tree visualization tools. It follows the binary search tree algorithm from the root (without animation) and reports whether the key was found or not. When the key is found, the node is encircled to highlight it.

The Delete Button

The Delete button acts like the one for the Binary Search Tree. First, it locates the requested key (without animation). If it's found, the number of children the node has determines how the deletion proceeds. With zero or one child, the parent's link to the node is adjusted. With two children, the successor node is found, copied to the node being deleted, and then the successor node is removed. The red-black rules are checked on the reduced tree to update the summary.

The Erase & Random Fill Button

When you want to start from an empty tree, type `0` in the text entry box and select the Erase & Random fill button. You can also create bigger (and more complex) trees by typing a larger number, up to 99. As the Binary Search Tree Visualization tool showed, you may not be able to insert keys in a sequence that fills all possible 31 nodes.

The red and black colors assigned to the nodes are not really random; their assignment follows the logic of individual insertions. Frequently, that will lead to at least one violation of a red-black rules when the tree size is three or more. Occasionally, the random filling satisfies all the rules.

Experimenting with the Visualization Tool

Now that you're familiar with the RedBlackTree operations, let's do some simple experiments to get a feel for what the tool does. The idea here is to learn to manipulate the tool's controls. Later you use these skills to balance the tree.

Experiment 1: Inserting Two Red Nodes

If there's already a tree, clear it by typing `0` in the text entry box and selecting Erase & Random Fill. Next, type `50` and select Insert. It's convenient to experiment with a root key of 50 because that number provides maximum flexibility to insert keys on either side. Not surprisingly, all the red-black rules are satisfied for this single node tree because the insert operation chooses black for root nodes.

Insert a second node with a value smaller than the root, say 20. The insert operation makes this node red, preventing any rule violations.

Insert a third node that's larger than the root, say 71. The new node is also red, and the tree remains red-black correct. It's also balanced. The result is shown in Figure 10-17.

Figure 10-17 *A balanced tree*

Notice that newly inserted nodes are always colored red (except for the root). This color is not an accident. Inserting a red node is less likely to violate the red-black rules than inserting a black one. The reason is that, if the new red node is attached to a black one, no rule is broken. The first insertion doesn't create a situation in which there are two red nodes together (Rule 3), and it doesn't change the black height in any of the paths (Rule 4). Of course, if you insert a new red node below a red node, Rule 3 will be violated. With any luck, however, this will happen only half the time. On the other hand, adding a new black node always changes the black height for its path, violating Rule 4 if the tree already satisfied that rule.

Also, it's easier to fix violations of Rule 3 (parent and child are both red) than Rule 4 (black heights differ), as you see later.

Experiment 2: Rotations

Let's try some rotations. Start with the three nodes shown on the left in Figure 10-18. Select node 50 as the top node in the rotation (by either typing the key in the text entry box or clicking it twice to select it without changing its color). Now perform a right rotation by selecting the Rotate right button. The nodes all shift to new positions, as shown in the right of Figure 10-18.

Figure 10-18 *The right rotation operation*

In this right rotation, the parent, or top, node moves into the place of its right child, the left child moves up and takes the place of the parent, and the right child moves down to become the grandchild of the new top node.

Notice that the tree is now unbalanced; the height of the right subtree is two more than that of the left. Also, the status messages indicate that the red-black rules are violated, specifically Rule 2 (the root is always black) and Rule 4 (black heights must be the same).

It's easy to get back to balance by rotating the other way. This time, you must select node 20 as the rotation top. Use the Rotate left button or hold the Shift key while double-clicking node 20 to rotate left. The nodes return to the position of Figure 10-17.

Experiment 3: Color Swaps

Start with the situation of Figure 10-17, with nodes 20 and 71 inserted in addition to 50 in the root position. Note that the parent (the root) is black, and both its children are red. Now try to insert another node, say with key value 12. No matter what value you use, you'll see a new kind of change. The animation shows the black color of the root being copied to the two red children. What's going on?

A color swap is necessary in the following situation: *when you're searching for the insertion point and encounter a black node with two red children*. If the two red children are leaves, it's clear why the colors must be flipped. Inserting a

new red node as a child of a red node would violate Rule 3. Flipping the color of the parent and both of its children avoids that problem.

Or course, if you swap the black root's color with its two red children, the root becomes red and violates Rule 2. When performing color swaps, we make an exception for the root node and leave it black.

Figure 10-19 shows the steps that happen while inserting a node with key value 12. As the internal `__find()` method descends the tree to locate the insertion point, it checks the current node's color and that of its children. In this case, it finds the black-red-red pattern at root node 50 and performs a swap (but keeps the root black). After it descends to node 20, it doesn't find the black-red-red pattern. The insertion goes in node 20's left child slot.

Figure 10-19 *Inserting a node with a color swap*

The tree remains red-black correct throughout the process. The root is black, there's no situation in which a parent and child are both red, and all the paths have the same number of black nodes (two). Adding the new red node doesn't change the red-black correctness.

Experiment 4: An Unbalanced Tree

Now let's see what happens when you try to do something that leads to an unbalanced tree. In the final tree of Figure 10-19, one path has one more node than the other. This example isn't very unbalanced and satisfies the definition of balance used with the AVL tree. No red-black rules are violated, so neither you nor the red-black algorithms need to worry about making changes.

Let's create an unbalanced tree. For this example, insert a node with key 6 into the final tree of Figure 10-19. You'll see the situation shown in Figure 10-20. Node 6 is red and lies beneath a red node 20. The red-red link is highlighted, and the status message shows the count of 1 for such links.

Figure 10-20 *Parent and child are both red*

How can you fix the tree so that Rule 3 isn't violated? An obvious approach is to change one of the indicated nodes to black. Try changing the child node 6 to black by clicking it.

The good news is you fixed the problem of both parent and child being red. The bad news is that now the status message for Rule 4 says Black heights: {2, 3}. The path from the root to node 6 has three black nodes in it, while the path from the root to node 71 has only two. It seems you can't win.

This problem, however, can be fixed with a rotation and some color changes. How to do this is the topic of later sections.

More Experiments

Experiment with the RedBlackTree Visualization tool on your own. Insert more nodes and see what happens. See whether you can use rotations and color changes to achieve a balanced tree. Does keeping the tree red-black correct seem to guarantee an (almost) balanced tree?

Try inserting five ascending keys (50, 60, 70, 80, 90) in an empty starting tree. Ignore the status messages until all the keys are inserted; then try flipping colors and rotating nodes. Can you balance the tree? If so, how many

operations did it take? Restart with an empty tree and try five descending keys (50, 40, 30, 20, 10). Can you balance this tree with fewer operations?

The Red-Black Rules and Balanced Trees

Try to create a tree that is unbalanced by two or more levels but is red-black correct. In other words, try to create a tree where at least one node has height difference of 3 or more. As it turns out, this is impossible. That's why the red-black rules keep the tree balanced. If one path is more than one node longer than another, it must either have more black nodes, violating Rule 4, or it must have two adjacent red nodes, violating Rule 3. Convince yourself that this is true by experimenting with the tool.

Null Children

Remember Rule 4, which specifies that all paths that go from the root to any leaf or to *any null children* must have the same number of black nodes. A null child is a child that a nonleaf (internal) node might have but doesn't. In other words, it's a missing left child if the internal node has a right child, or vice versa. In Figure 10-21 the path from 50 to 20 to the right child of 20 (its null child) has two black nodes in the tree on top. The path from 50 to 20 to 12 has three, which violates Rule 4. The paths to both leaf nodes have the same number of black nodes, but it's the null children higher up that cause the problem. Even if those internal nodes are red, as shown in the tree on the bottom, the number of black nodes in the four paths don't match.

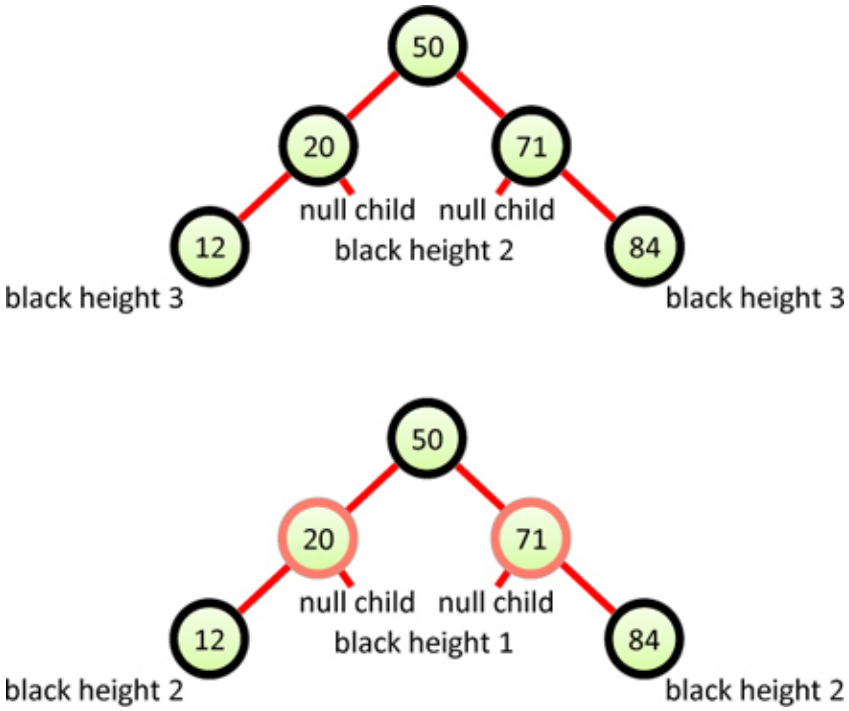

Figure 10-21 *Paths to null children*

Rotations in Red-Black Trees

To balance a tree, some algorithm must rearrange the nodes. If too many nodes are on the left of the root, as in Figure 10-20 for example, you need to move some of them over to the right side. As with the other binary trees, this is done using rotations. In this section you learn how rotations interact with the red-black rules.

Note that red-black rules and node colors are used only to help decide when to perform a rotation. Fiddling with the colors doesn't accomplish anything by itself; it's the rotation that's the heavy hitter. It's something like making improvements to your house or apartment. Changing the colors of walls and adding plants can change the mood of the room, while moving walls and adding doorways changes the whole structure.

Subtrees on the Move

You've seen individual nodes changing position during a rotation, but as you saw with AVL trees, entire subtrees can move as well. To see this movement, start with an empty tree (by entering `0` and selecting Erase & Random Fill), and

then insert the following sequence of nodes in order: 50, 25, 75, 12, 37, 62, 87, 6, 18, 31, 43. You will see color swaps occur when you insert nodes 12 and 6. The resulting arrangement is shown on the left of Figure 10-22.

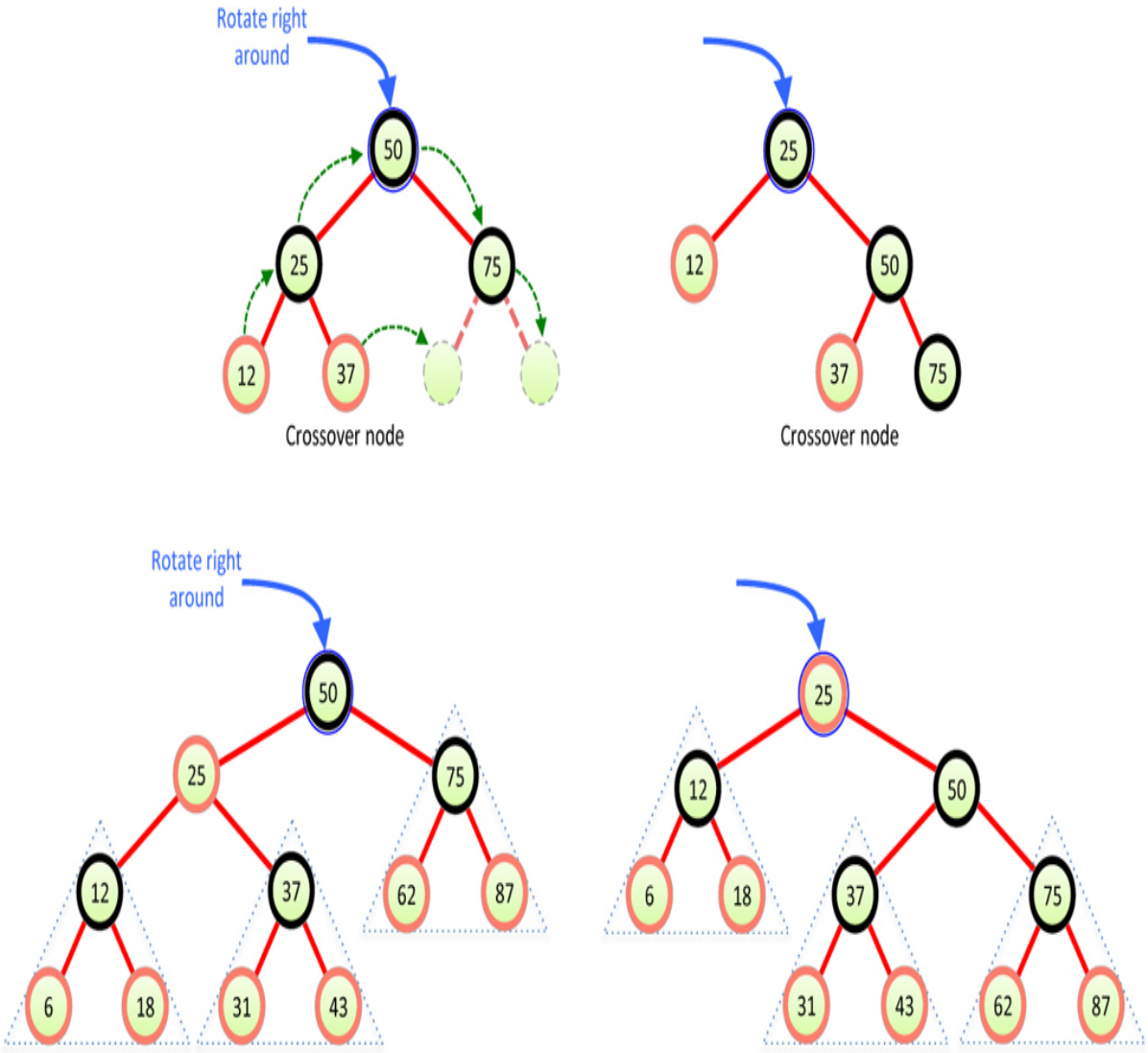

Figure 10-22 *Subtree movement during rotation*

Now rotate around root node 50 by double-clicking it or typing `50` and selecting the Rotate Right button. A lot of nodes have changed position. The result is shown on the right of Figure 10-22. Here's what happens:

- The top node (50) goes to its right child.
- The top node's left child (25) goes to the top (root).